LOOPS

Essencialmente, os loops são uma forma de fazer algo repetidamente.

Comece digitando o código cat.py na janela do terminal.

No editor de texto, comece com o seguinte código:

```
print("meow")
print("meow")
print("meow")
```

Executando este código digitando python cat.py, você notará que o programa mia três vezes.

Ao se desenvolver como programador, você deve considerar como alguém poderia melhorar áreas do seu código onde se digita a mesma coisa repetidamente. Imagine onde alguém poderia querer “miar” 500 vezes. Seria lógico digitar a mesma expressão print("meow") repetidamente?

Os loops permitem criar um bloco de código que é executado continuamente.

WHILE LOOPS (ENQUANTO FAZ LOOPS)

O loop while é quase universal em todas as linguagens de codificação.

Esse loop repetirá um bloco de código indefinidamente.

Na janela do editor de texto, edite seu código da seguinte maneira:

```
i = 3
while i != 0:
  print("meow")
```

Observe como mesmo que este código execute print("meow") várias vezes, ele nunca irá parar! Ele fará um loop para sempre. Os loops while funcionam perguntando repetidamente se a condição do loop foi cumprida. Nesse caso, o compilador está perguntando: “não é igual a zero?” Quando você fica preso em um loop que é executado para sempre, você pode pressionar control-c no teclado para sair do loop.

Para corrigir esse loop que dura para sempre, podemos editar nosso código da seguinte maneira

```
i = 3
```

```
while i != 0:
 print("meow")
 i = i - 1
```

Observe que agora nosso código é executado corretamente, reduzindo i em 1 para cada “iteração” do loop. O termo iteração tem um significado especial na codificação. Por iteração, queremos dizer um ciclo no loop. A primeira iteração é a “0ª” iteração através do loop. A segunda é a “1ª” iteração. Na programação, contamos começando com 0, depois 1 e depois 2.

Podemos melhorar ainda mais nosso código da seguinte maneira:

```
i = 1
 while i <= 3:
   print("meow")
   i = i + 1
```

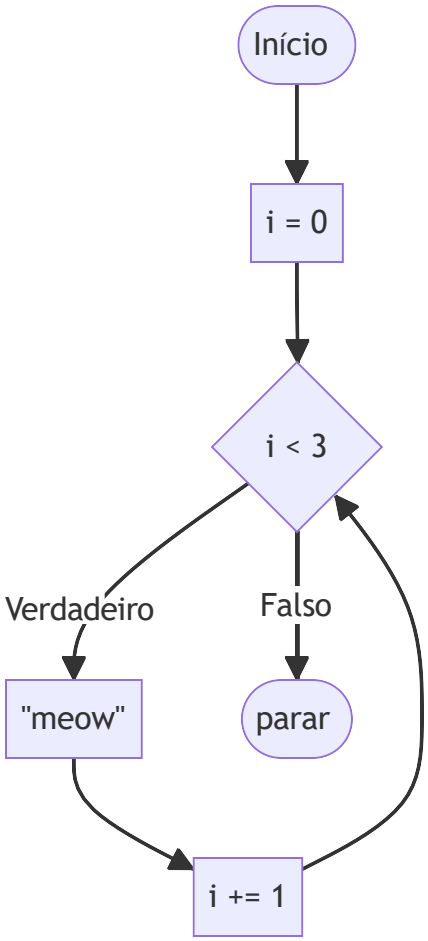
Observe que quando codificamos i = i + 1 atribuímos o valor de i da direita para a esquerda. Acima, estamos começando com um, como a maioria dos humanos conta (1, 2, 3). Se você executar o código acima, verá que ele mia três vezes. É uma prática recomendada em programação começar a contar com zero.

Podemos melhorar nosso código para começar a contar do zero:

```
i = 0
while i < 3:
  print("meow")
  i += 1
```

Observe como alterar o operador para i <3 permite que nosso código funcione conforme planejado. Começamos contando com 0 e ele percorre nosso loop três vezes, produzindo três miados. Além disso, observe como i += 1 é o mesmo que dizer i = i + 1.

Nosso código neste ponto é ilustrado da seguinte forma:



Observe como nosso loop conta até, mas não até 3.

FOR LOOPS

Um loop for é um tipo diferente de loop.

Para entender melhor um loop for, é melhor começar falando sobre um novo tipo de variável chamado lista em Python. Como em outras áreas da nossa vida, podemos ter uma lista de compras, uma lista de tarefas, etc.

Um loop for percorre uma lista de itens. Por exemplo, na janela do editor de texto, modifique seu código cat.py da seguinte forma:

```
for i in [0, 1, 2]:
    print("meow")
```

Observe como esse código é limpo em comparação com o código do loop while anterior. Neste código, i começa com 0, mia, i recebe 1, mia e, finalmente, i recebe 2, mia e termina.

Embora esse código atinja o que desejamos, existem algumas possibilidades de melhorá-lo em casos extremos. À primeira vista, nosso código parece ótimo. No entanto, e se você quisesse iterar até um milhão? É melhor criar um código que funcione com esses casos extremos. Conseqüentemente, podemos melhorar nosso código da seguinte forma:

```
for i in range(3):
    print("meow")
```

Observe como range(3) retorna três valores (0, 1 e 2) automaticamente. Este código irá executar e produzir o efeito pretendido, miando três vezes.

Nosso código pode ser melhorado ainda mais. Observe como nunca usamos i explicitamente em nosso código. Ou seja, embora o Python precise do i como um local para armazenar o número da iteração do loop, nunca o usamos para qualquer outro propósito. Em Python, se tal variável não tiver nenhum outro significado em nosso código, podemos simplesmente representar essa variável como um único sublinhado _. Portanto, você pode modificar seu código da seguinte maneira:

```
for _ in range(3):
    print("meow")
```

Observe como mudar i para _ tem impacto zero no funcionamento do nosso programa.

Nosso código pode ser melhorado ainda mais. Para ampliar sua mente para as possibilidades do Python, considere o seguinte código:

```
print("meow" * 3)
```

Observe como ele mia três vezes, mas o programa produzirá miau miau como resultado. Considere: como você poderia criar uma quebra de linha no final de cada miado?

Na verdade, você pode editar seu código da seguinte maneira:

```
print("meow\n" * 3, end="")
```

Observe como esse código produz três miados, cada um em uma linha separada. Adicionando end="" e \n dizemos ao compilador para adicionar uma quebra de linha no final de cada miado.

MELHORANDO A ENTRADA DO USUÁRIO

Talvez queiramos obter informações de nosso usuário. Podemos usar loops como forma de validar a entrada do usuário.

Um paradigma comum em Python é usar um loop while para validar a entrada do usuário.

Por exemplo, vamos tentar solicitar ao usuário um número maior ou igual a 0:

```
while True:
  n = int(input("What's n? "))
  if n < 0:
    continue
  else:
    break
```

Observe que introduzimos duas novas palavras-chave em Python, continue e break. continue diz explicitamente ao Python para ir para a próxima iteração de um loop. break, por outro lado, diz ao Python para “sair” de um loop antes de terminar todas as suas iterações. Nesse caso, continuaremos para a próxima iteração do loop quando n for menor que 0 – no final, perguntando novamente ao usuário “O que é n?”. Se, porém, n for maior ou igual a 0, sairemos do loop e permitiremos que o resto do nosso programa seja executado.

Acontece que a palavra-chave continue é redundante neste caso. Podemos melhorar nosso código da seguinte maneira:

```
while True:
  n = int(input("What's n? "))
  if n > 0:
    break
```

```
for _ in range(n):
  print("meow")
```

Observe como esse loop while sempre será executado (para sempre) até que n seja maior que 0. Quando n for maior que 0, o loop será interrompido.

Trazendo nosso aprendizado anterior, podemos usar funções para melhorar ainda mais nosso código:

```
def main():
  meow(get_number())



def get_number():
  while True:
    n = int(input("What's n? "))
    if n > 1:
      return n



def meow(n):
  for _ in range(n):
    print("meow")



main()
```

Observe como não apenas alteramos seu código para operar em múltiplas funções, mas também usamos uma instrução return para retornar o valor de n de volta à função principal.

MAIS SOBRE LISTAS

Considere o mundo de Hogwarts do famoso universo Harry Potter.

No terminal, digite o código hogwarts.py.

No editor de texto, codifique da seguinte forma:

```
students = ["Hermoine", "Harry", "Ron"]


print(students[0])
print(students[1])
```

```
print(students[2])
```

Observe como temos uma lista de alunos com seus nomes conforme acima. Imprimimos então a aluna que está no local 0, “Hermoine”. Cada um dos outros alunos também é impresso.

Assim como ilustramos anteriormente, podemos usar um loop para iterar na lista. Você pode melhorar seu código da seguinte maneira:

```
students = ["Hermoine", "Harry", "Ron"]

for student in students:
  print(student)
```

Observe que para cada aluno na lista de alunos, o aluno será impresso conforme pretendido. Você pode estar se perguntando por que não usamos a designação _ conforme discutido anteriormente. Optamos por não fazer isso porque student é explicitamente usado em nosso código.

COMPRIMENTO

Podemos utilizar *len* como forma de verificar o comprimento da lista chamada de alunos.

Imagine que você não deseja imprimir apenas o nome do aluno, mas também sua posição na lista. Para fazer isso, você pode editar seu código da seguinte maneira:

```
students = ["Hermoine", "Harry", "Ron"]

for i in range(len(students)):
  print(i + 1, students[i])
```

Observe como a execução desse código resulta não apenas na obtenção da posição de cada aluno mais um usando i + 1, mas também na impressão do nome de cada aluno. len permite que você veja dinamicamente o tamanho da lista de alunos, independentemente de quanto ela cresce.

DICIONÁRIOS

Dicionários é uma estrutura de dados que permite associar chaves a valores.

Onde uma lista é uma lista de vários valores, um dict associa uma chave a um valor.

Considerando as casas de Hogwarts, poderíamos atribuir alunos específicos a casas específicas.

Hermione	Harry	Ron	Draco
Gryffindor	Gryffindor	Gryffindor	Slytherin

Poderíamos usar apenas listas para fazer isso:

```
students = ["Hermoine", "Harry", "Ron", "Draco"]
houses = ["Gryffindor", "Gryffindor", "Griffindor", "Slytherin"]
```

Observe que podemos prometer que sempre manteremos essas listas em ordem. O indivíduo que ocupa a primeira posição de alunos é associado à casa que ocupa a primeira posição da lista de casas, e assim sucessivamente.

No entanto, isso pode se tornar bastante complicado à medida que nossas listas crescem!

Podemos melhorar nosso código usando um ditado da seguinte maneira:

```
students = {
  "Hermoine": "Gryffindor",
  "Harry": "Gryffindor",
  "Ron": "Gryffindor",
  "Draco": "Slytherin",
}
print(students["Hermoine"])
```

```
print(students["Harry"])
print(students["Ron"])
print(students["Draco"])
```

Observe como usamos {} chaves para criar um dicionário. Enquanto as listas usam números para percorrer a lista, os dictos nos permitem usar palavras.

Execute seu código e certifique-se de que sua saída seja a seguinte:

```
$ python hogwarts.py
Gryffindor
Gryffindor
Gryffindor
Slytherin
```

Podemos melhorar nosso código da seguinte maneira:

```
students = {
  "Hermoine": "Gryffindor",
  "Harry": "Gryffindor",
  "Ron": "Gryffindor",
  "Draco": "Slytherin",
}
for student in students:
  print(student)
```

Observe como, ao executar este código, o loop for irá apenas iterar por todas as chaves, resultando em uma lista com os nomes dos alunos. Como poderíamos imprimir valores e chaves?

Modifique seu código da seguinte maneira:

```
students = {
```

```
    "Hermoine": "Gryffindor",
    "Harry": "Gryffindor",
    "Ron": "Gryffindor",
    "Draco": "Slytherin",
}
for student in students:
    print(student, students[student])
```

Observe como os alunos irão até a chave de cada aluno e descobrirão o valor de sua casa. Execute seu código e você notará como a saída fica um pouco confusa.

Podemos limpar a função de impressão melhorando nosso código da seguinte forma:

```
students = {
    "Hermoine": "Gryffindor",
    "Harry": "Gryffindor",
    "Ron": "Gryffindor",
    "Draco": "Slytherin",
}
for student in students:
    print(student, students[student], sep=", ")
```

Observe como isso cria uma separação clara de a entre cada item impresso.

Se você executar python hogwarts.py, deverá ver o seguinte:

```
$ python hogwarts.py
Hermoine, Gryffindor
Harry, Gryffindor
Ron, Gryffindor
Draco, Slytherin
```

E se tivermos mais informações sobre nossos alunos? Como poderíamos associar mais dados a cada um dos alunos?

	name	house	patronus
0	Hermione	Gryffindor	Otter
1	Harry	Gryffindor	Stag
2	Ron	Gryffindor	Jack Russell terrier
3	Draco	Slytherin	

Você pode imaginar querer ter muitos dados associados a múltiplas chaves. Aprimore seu código da seguinte maneira:

```
students = [
  {"name": "Hermoine", "house": "Gryffindor", "patronus": "Otter"},
  {"name": "Harry", "house": "Gryffindor", "patronus": "Stag"},
  {"name": "Ron", "house": "Gryffindor", "patronus": "Jack Russell terrier"},
  {"name": "Draco", "house": "Slytherin", "patronus": None},
]
```

Observe como este código cria uma lista de dictos. A lista chamada de alunos contém quatro dictos: Um para cada aluno. Além disso, observe que Python tem uma designação especial None, onde não há valor associado a uma chave.

Agora você tem acesso a uma série de dados interessantes sobre esses alunos. Agora, modifique ainda mais seu código da seguinte maneira:

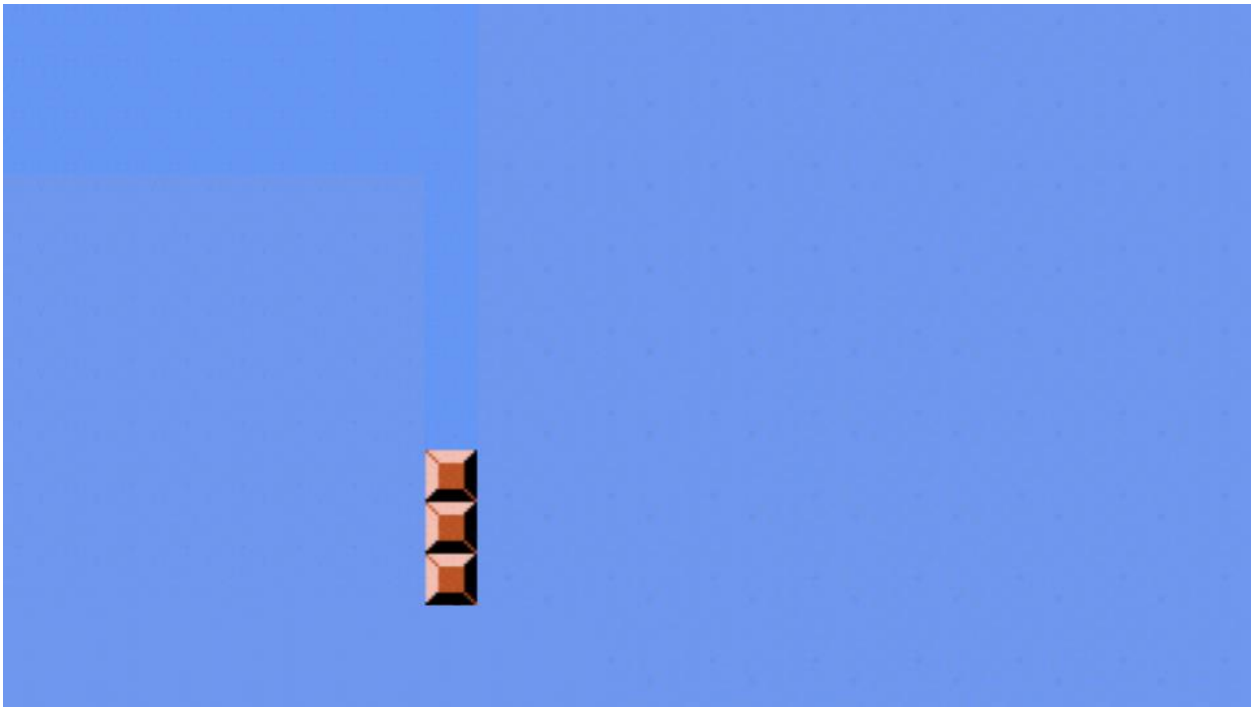
```
students = [
  {"name": "Hermoine", "house": "Gryffindor", "patronus": "Otter"},
  {"name": "Harry", "house": "Gryffindor", "patronus": "Stag"},
  {"name": "Ron", "house": "Gryffindor", "patronus": "Jack Russell terrier"},
```

```
    {"name": "Draco", "house": "Slytherin", "patronus": None},
]

for student in students:
    print(student["name"], student["house"], student["patronus"], sep=", ")
```

MARIO

Lembre-se que o clássico jogo Mario tem um herói pulando tijolos. Vamos criar uma representação textual deste jogo.



Comece a codificação da seguinte maneira:

```
print("#")
print("#")
print("#")
```

Observe como copiamos e colamos o mesmo código repetidamente.

Considere como poderíamos melhorar o código da seguinte maneira:

```
for _ in range(3):
    print("#")
```

Observe como isso realiza essencialmente o que queremos criar.

Considere: poderíamos abstrair ainda mais para resolver problemas mais sofisticados posteriormente com este código? Modifique seu código da seguinte maneira:

```
def main():
    print_column(3)


def print_column(height):
    for _ in range(height):
        print("#")


main()
```

Observe como nossa coluna pode crescer tanto quanto quisermos, sem qualquer codificação.

Agora, vamos tentar imprimir uma linha horizontalmente. Modifique seu código da seguinte maneira:

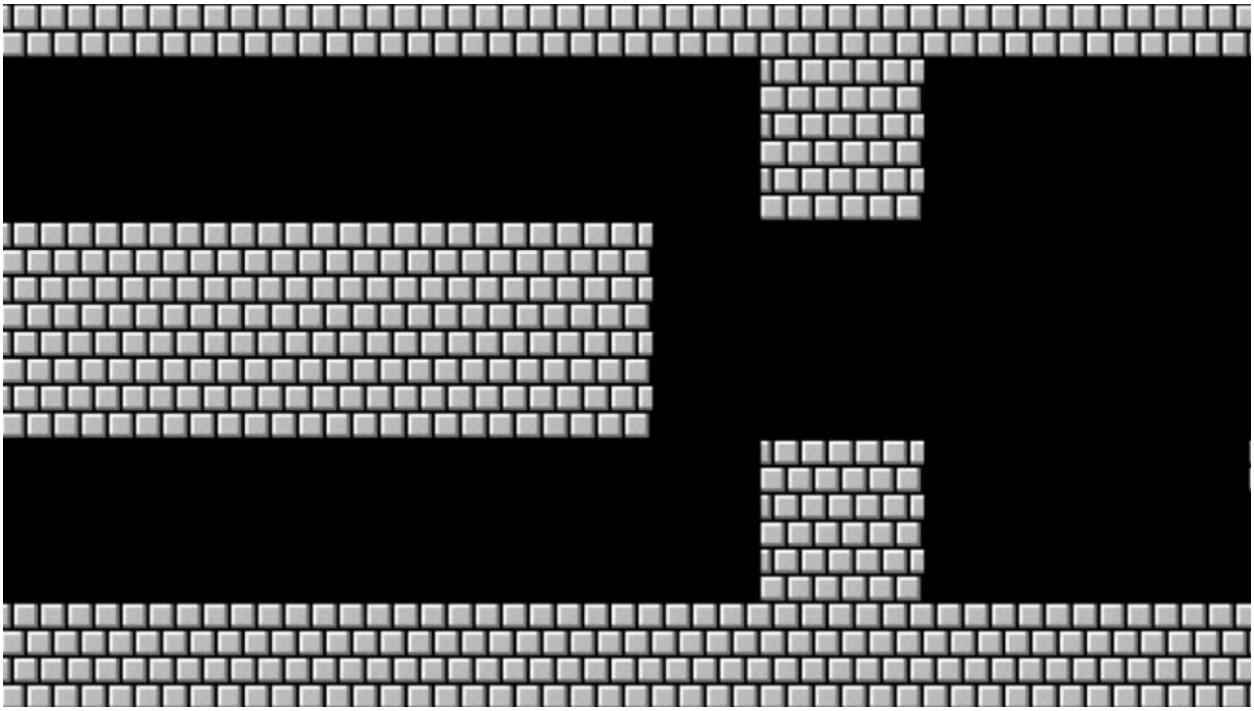
```
def main():
    print_row(4)


def print_row(width):
    print("?" * width)
```

```
main()
```

Observe como agora temos código que pode criar blocos da esquerda para a direita.

Examinando o slide abaixo, observe como Mario possui linhas e colunas de blocos.



Considere: como poderíamos implementar linhas e colunas em nosso código? Modifique seu código da seguinte maneira:

```
def main():
    print_square(3)


def print_square(size):

    # For each row in square
    for i in range(size):
```

```
    # For each brick in row
    for j in range(size):

        #  Print brick
        print("#", end="")

    # Print blank line
    print()



main()
```

Observe que temos um loop externo que aborda cada linha do quadrado. Então, temos um loop interno que imprime um tijolo em cada linha. Finalmente, temos uma instrução print que imprime uma linha em branco.

Podemos abstrair ainda mais nosso código:

```
def main():
    print_square(3)



def print_square(size):
    for i in range(size):
        print_row(size)



def print_row(width):
    print("#" * width)



main()
```

RESUMINDO

Agora você tem outro poder em sua lista crescente de habilidades em Python. Nesta palestra abordamos…

- Loops
- while
- for
- len
- list
- dict